SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4316

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

Code of Conduct:

I P.Nikitha/192324226 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

QUESTIONS: **Total Marks: 20**

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.
(a) Identify key components required in a navigation bar for an e-commerce website.
(b) Design a responsive navigation bar using appropriate CSS techniques.
(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
(d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.
(a) Identify suitable CSS properties for hover effects.
(b) Suggest appropriate **CSS animation or transition** for product interaction.
(c) Design a hover effect for a product card.
(d) Evaluate performance and user experience considerations.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.
(a) Identify layout requirements of a dashboard (rows, columns, alignment).
(b) Compare **Flexbox and Grid** based on layout needs.
(c) Choose the most suitable layout technique and justify your choice.
(d) Evaluate the scalability and responsiveness of the chosen approach.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.
(a) Identify key sections of a blog layout.
(b) Design a mobile-first layout using CSS.
(c) Demonstrate how the layout scales for larger screens using media queries.
(d) Evaluate accessibility and readability of the design.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.
(a) Identify appropriate JavaScript events for menu interaction.
(b) Design event handling logic for menu toggle functionality.
(c) Implement interaction for user actions (click/hover).
(d) Evaluate performance and suggest improvements.

Question 1:

Scenario: An e-commerce website requires a responsive navigation bar that adapts across desktop, tablet and mobile devices.

(a) Identify key components required in a navigation bar.

A navigation bar is an important part of a website that helps users move between different pages. The key components are:

1. Logo: Represents the website brand.
2. Home: Redirects users to the homepage.
3. Search bar: Helps users search for products.
4. Categories: Displays different product categories.
5. Account: Allows users to log in or manage their profiles.
6. Shopping Cart: Displays selected products.
7. Hamburger Menu: Displays navigation links on smaller screens.

(b) Design a responsive navigation bar using CSS.

We can use HTML to create the navigation structure and CSS Flexbox to arrange the elements.

HTML Code:

```
<!DOCTYPE html>
<html>
<head>
<title>Navigation Bar</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<nav class="navbar">

<h2>ShopEasy</h2>

<div class="menu">
<a href="#">Home</a>
<a href="#">Products</a>
<a href="#">Offers</a>
<a href="#">Account</a>
<a href="#">Cart</a>
</div>

</nav>

</body>
</html>
```

CSS Code:

```
.navbar {
   display: flex;
   justify-content: space-between;
   align-items: center;
   background: navy;
   padding: 15px;
   color: white;
}

.menu {
   display: flex;
   gap: 20px;
}

.menu a {
   color: white;
   text-decoration: none;
}

@media(max-width:600px) {
   .menu {
      flex-direction: column;
      display: none;
   }

   .menu.active {
      display: flex;
   }
}
```

(c) Demonstrate how the navigation adapts to smaller screens.

For smaller screens, a hamburger menu is used to display or hide navigation links.

JavaScript Code:

```
<button onclick="toggleMenu()">☰</button>

<script>
function toggleMenu() {
   document.querySelector(".menu")
   .classList.toggle("active");
}
```

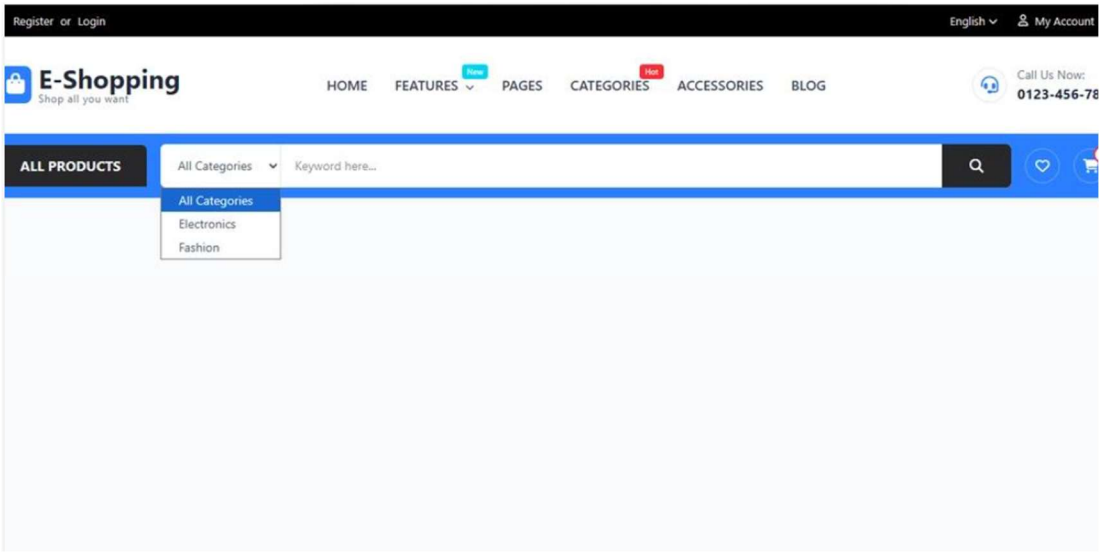
</script>
Explanation:

- On desktop, all navigation links appear horizontally.
- On mobile, the navigation links are hidden initially.
- When the user clicks the hamburger icon, the menu becomes visible.
- CSS media queries control the layout according to screen size.

(d) Evaluate usability and suggest improvements.

- Use clear labels for navigation links.
- Make the search bar easily accessible.
- Use sufficient color contrast.
- Make buttons large enough for mobile users.
- Ensure the menu works with keyboard navigation.
- Use responsive layouts for different devices.

Conclusion: A responsive navigation bar improves website usability by providing easy navigation across desktop, tablet and mobile devices.



Question 2:

Scenario: An e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Identify suitable CSS properties for hover effects.

CSS provides several properties to create attractive hover effects.

1. transform – Moves, rotates or scales an element.
2. transition – Creates smooth changes between styles.
3. box-shadow – Adds shadow effects.
4. opacity – Changes the transparency.
5. background-color – Changes the background color.
6. cursor – Changes the mouse pointer.

(b) Suggest appropriate CSS animation or transition.

The following CSS properties can be used:

```
.product {
   transition: 0.3s ease;
}

.product:hover {
   transform: scale(1.05);
   box-shadow: 0 5px 15px gray;
}
```

Explanation:

- transition: 0.3s ease creates a smooth effect.
- scale(1.05) slightly increases the product size.
- box-shadow adds a shadow when the mouse moves over the product.

(c) Design a hover effect for a product card.

HTML Code:

```
<!DOCTYPE html>
<html>
<head>
<title>Product Card</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="product">

<img src="shoe.jpg" width="150">

<h3>Running Shoes</h3>

<p>Price: ₹1499</p>

<button>Buy Now</button>

</div>

</body>
</html>
```

CSS Code:

```
.product {
   width: 200px;
```

```
    padding: 15px;
    text-align: center;
    border: 1px solid gray;
    transition: 0.3s;
}

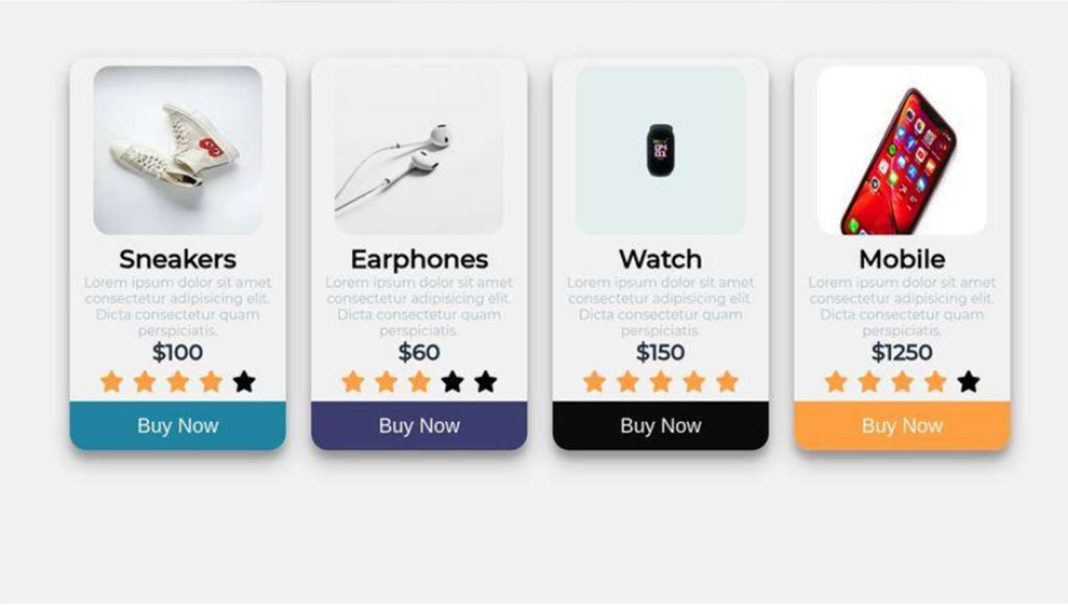
.product:hover {
    transform: translateY(-10px);
    box-shadow: 0 8px 15px gray;
}

button {
    background: blue;
    color: white;
    padding: 10px;
    border: none;
}
```

(d) Evaluate performance and user experience considerations.

- Use simple hover animations.
- Avoid excessive animations.
- Use smooth transitions.
- Avoid large images that slow down the website.
- Provide alternative visual feedback for touchscreen users.
- Use prefers-reduced-motion to respect users who prefer reduced animations.

Conclusion: CSS hover effects improve the visual appearance of product cards and make an e-commerce website more interactive and attractive.



Question 3:

Scenario: A dashboard layout needs to be designed for a web application.

(a) Identify layout requirements of a dashboard.

A dashboard displays important information in an organized manner.

The main requirements are:

1. Rows: Arrange content horizontally.
2. Columns: Divide the dashboard into different sections.
3. Alignment: Align elements properly.
4. Spacing: Maintain equal gaps between elements.
5. Responsiveness: Adjust the layout according to screen size.
6. Consistency: Maintain a uniform design.

(b) Compare Flexbox and Grid based on layout needs.

CSS provides two important layout techniques: Flexbox and Grid.

Flexbox	**CSS Grid**
One-dimensional layout	Two-dimensional layout
Works with rows or columns	Works with rows and columns
Best for navigation bars	Best for dashboards
Aligns items along one axis	Controls rows and columns together
Suitable for small components	Suitable for complex layouts

(c) Choose the most suitable layout technique and justify your choice.

CSS Grid is the most suitable technique for dashboard design.

Reason:

- A dashboard contains multiple rows and columns.
- Grid provides better control over element positioning.
- It maintains equal spacing between cards.
- It supports responsive layouts.
- It makes complex layouts easier to manage.

HTML Code:

```
<div class="dashboard">

<div class="card">Total Sales</div>

<div class="card">Customers</div>

<div class="card">Orders</div>

<div class="card">Products</div>

</div>
```

CSS Code:

```
.dashboard {
   display: grid;
   grid-template-columns: repeat(3, 1fr);
   gap: 20px;
   padding: 20px;
}

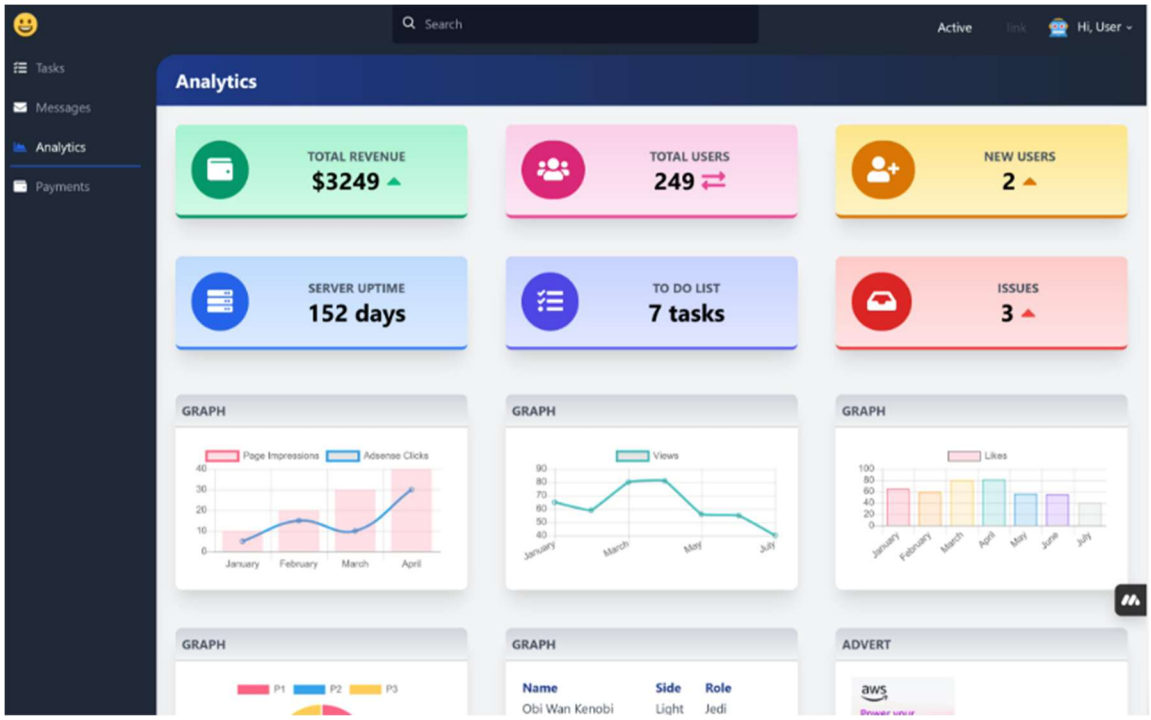
.card {
   background: lightblue;
   padding: 30px;
   text-align: center;
   border-radius: 10px;
}

@media(max-width:600px) {
   .dashboard {
      grid-template-columns: 1fr;
   }
}
```

(d) Evaluate scalability and responsiveness.

- Grid makes it easy to add new dashboard cards.
- Media queries adjust the number of columns.
- It supports different screen sizes.
- Equal spacing improves the appearance.
- It reduces the need for complicated positioning.

Conclusion: CSS Grid is the best choice for dashboard layouts because it provides flexible two-dimensional control and makes the design scalable and responsive.



Question 4:

Scenario: A blog page must be designed following a mobile-first approach.

(a) Identify key sections of a blog layout.

A blog layout contains the following sections:

1. Header: Displays the blog title or logo.
2. Navigation Bar: Provides links to different pages.
3. Main Content: Displays blog articles.
4. Sidebar: Displays related posts and categories.
5. Footer: Contains copyright and contact information.

(b) Design a mobile-first layout using CSS.

Mobile-first design means developing the website for smaller screens first and then adjusting it for larger screens.

HTML Code:

```
<!DOCTYPE html>
<html>
<head>
<title>My Blog</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<header>
<h2>My Blog</h2>
</header>
```

```
<nav>
Home | About | Contact
</nav>

<main>

<article>
<h3>Technology Trends</h3>
<p>Technology is changing
the world rapidly.</p>
</article>

<aside>
<h3>Related Posts</h3>
<p>AI Technology</p>
<p>Web Development</p>
</aside>

</main>

<footer>
<p>Copyright 2026</p>
</footer>

</body>
</html>
```

CSS Code:

```
body {
   margin: 0;
   font-family: Arial;
}

header, nav, footer {
   background: navy;
   color: white;
   padding: 15px;
   text-align: center;
}
```

```
main {
   display: flex;
   flex-direction: column;
   padding: 15px;
}

article, aside {
   background: lightgray;
   padding: 15px;
   margin: 10px;
}
```

(c) Demonstrate how the layout scales for larger screens using media queries.

We use media queries to change the layout when the screen width increases.

CSS Code:

```
@media(min-width:768px) {

   main {
      flex-direction: row;
   }

   article {
      flex: 3;
   }

   aside {
      flex: 1;
   }

}
```

Explanation:

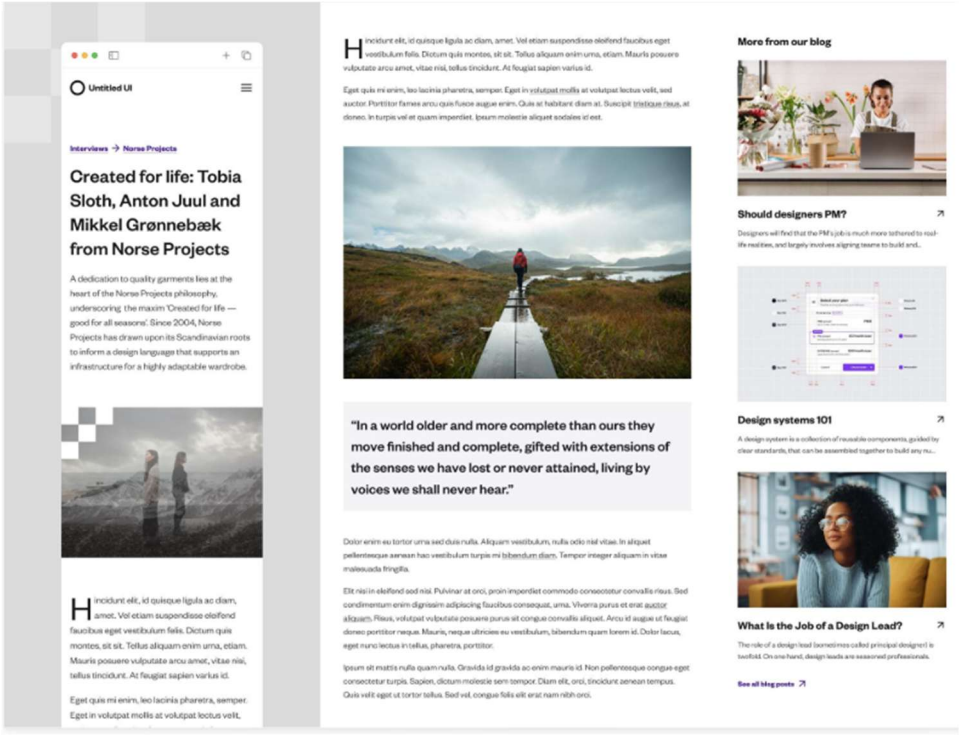
- On mobile screens, the article and sidebar appear vertically.
- On tablet and desktop screens, they appear side by side.
- The article gets more space than the sidebar.
- Media queries help the design adapt to different screen sizes.

(d) Evaluate accessibility and readability.

- Use readable font sizes.
- Provide sufficient color contrast.
- Use proper headings.
- Add alternative text to images.
- Use semantic HTML elements.

- Maintain proper spacing between paragraphs.
- Make navigation keyboard accessible.

Conclusion: A mobile-first blog layout improves readability and accessibility while providing a consistent experience across different devices.



Question 5:

Scenario: A website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events for menu interaction.

JavaScript events are actions performed by users or browsers.

Important events include:

1. click: Executes an action when the user clicks an element.
2. mouseover: Executes when the mouse moves over an element.
3. mouseout: Executes when the mouse leaves an element.
4. keydown: Executes when a keyboard key is pressed.
5. load: Executes when a webpage finishes loading.

(b) Design event handling logic for menu toggle functionality.

A dynamic menu can be created using JavaScript click events.

The addEventListener() method is used to attach an event to an HTML element.

HTML Code:

```
<button id="menuBtn">Menu</button>

<div id="menu">
<a href="#">Home</a>
<a href="#">About</a>
```

```
<a href="#">Contact</a>
</div>
```

(c) Implement interaction for user actions (click/hover).

CSS Code:

```
#menu {
   display: none;
   background: lightblue;
   padding: 15px;
}

#menu a {
   display: block;
   padding: 5px;
}
```

JavaScript Code:

```
let button = document.getElementById("menuBtn");

let menu = document.getElementById("menu");

button.addEventListener("click", function() {

   if(menu.style.display == "none") {
      menu.style.display = "block";
   }

   else {
      menu.style.display = "none";
   }

});
```

Explanation:

1. First, JavaScript selects the menu button and menu section.
2. addEventListener() listens for a click event.
3. When the user clicks the button, the menu visibility is checked.
4. If the menu is hidden, it becomes visible.
5. If the menu is visible, it becomes hidden.

For hover interaction, we can use:

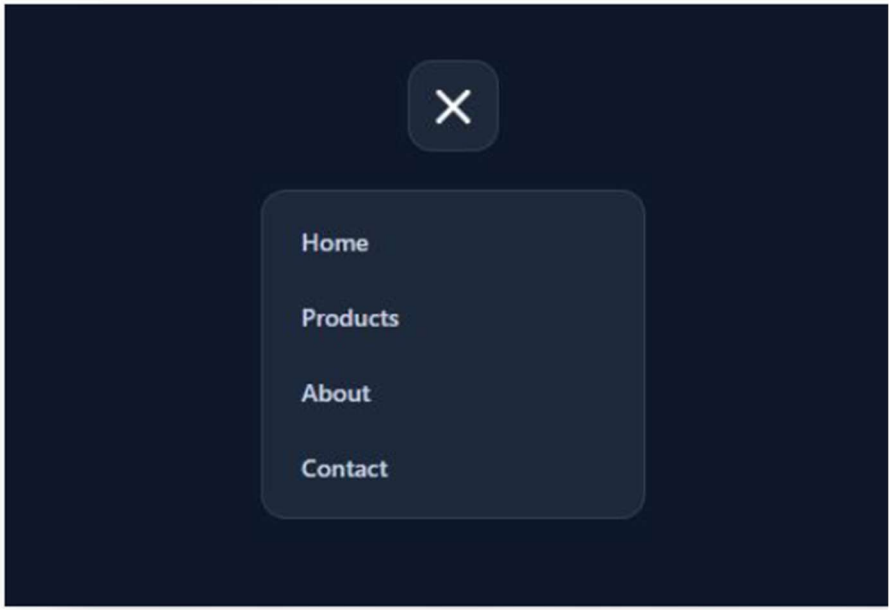
```
button.addEventListener("mouseover", function() {
   menu.style.display = "block";
});
```

```
button.addEventListener("mouseout", function() {
   menu.style.display = "none";
});
```

(d) Evaluate performance and suggest improvements.

- Use addEventListener() for event handling.
- Avoid unnecessary event listeners.
- Use CSS for simple visual effects.
- Use event delegation for multiple menu items.
- Ensure the menu works on mobile devices.
- Support keyboard interactions.
- Update aria-expanded to improve accessibility.

Conclusion: JavaScript event handling makes a website interactive by allowing menus to respond dynamically to user actions such as clicking and hovering.



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation